

Kubernetes Real-World Projects

20 HANDS-ON PROJECTS FOR JUNIOR ENGINEERS

BUILD | DEPLOY | SECURE | MONITOR | TROUBLESHOOT | OPERATE

-  Containerized Applications
-  Deployments & Services
-  Ingress & TLS
-  Storage & Databases
-  Scaling & High Availability
-  Security & RBAC
-  CI/CD & GitOps
-  Monitoring & Logging
-  Troubleshooting
-  Production-Style Project



*From
Learning
to
Production*



REAL PROJECTS
REAL SKILLS
REAL ENGINEERING

A PRACTICAL NOTEBOOK
FOR FUTURE-READY ENGINEERS

VERIQTA
ENGINEER TODAY
A HIGHER TOMORROW

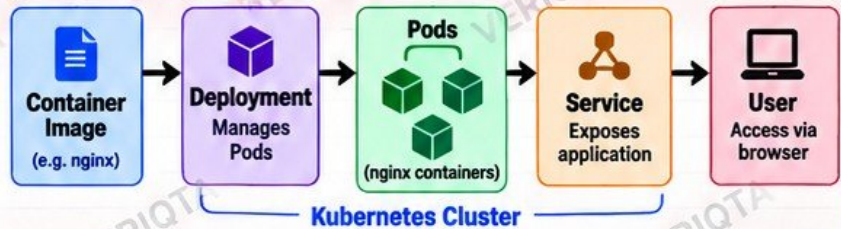
Deploy Your First Real Application

Containerized web application deployed with a Deployment and exposed with a Service. Learn the path from container image to running application.

WHAT YOU WILL BUILD

- Deploy a real web application (Nginx).
- Use a Deployment to manage Pods.
- Expose the application with a Service.
- Access the application from your browser.
- Understand how a container image becomes a running application in Kubernetes.

ARCHITECTURE OVERVIEW



1. CREATE A DEPLOYMENT

Create a file named `nginx-deployment.yaml`

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-app
  template:
    metadata:
      labels:
        app: nginx-app
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
  
```

2. CREATE A SERVICE

Create a file named `nginx-service.yaml`

```

apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort
  selector:
    app: nginx-app
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
  
```

3. DEPLOY TO KUBERNETES

Apply the manifests:

```

kubectl apply -f nginx-deployment.yaml
kubectl apply -f nginx-service.yaml
  
```

4. VERIFY THE DEPLOYMENT

Check that everything is running:

```

kubectl get deployments
kubectl get pods
kubectl get services
  
```

5. ACCESS THE APPLICATION

Get your node IP and open in browser:

```

kubectl get nodes -o wide
http://<Node-IP>:30080
  
```

You should see the Nginx welcome page.

KEY KUBERNETES CONCEPTS

- Container Image** - A packaged application.
- Deployment** - Manages the desired state.
- Pod** - Runs the container.
- Service** - Exposes the application.
- NodePort** - Makes the app accessible from outside the cluster.

COMMON BEGINNER MISTAKES

- ✗ Forgetting to set the correct port.
- ✗ Using the wrong image name.
- ✗ Pods not running due to image pull errors.
- ✗ Confusing Pod IP with Service IP.
- ✗ Expecting the app to be accessible without a Service.

PRACTICE THIS

- ✓ Change the number of replicas to 2.
- ✓ Update the Nginx image to a specific version (e.g. 1.25).
- ✓ Try deleting a Pod and observe the replacement.
- ✓ Change the Service type to ClusterIP and test internal access.



JUNIOR ENGINEER TIP

You do not manage individual Pods. You declare the desired state with a Deployment, and Kubernetes ensures the application keeps running.



NEXT STEP

In the next project, you will build a multi-tier application with a frontend, backend and database.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Build a Multi-Tier Application

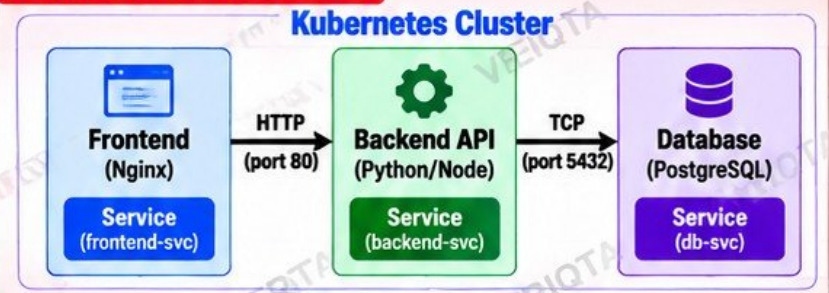
Frontend → backend API → database

Understand how separate workloads communicate inside a cluster.

WHAT YOU WILL BUILD

- ✓ A simple 3-tier application.
- ✓ Frontend (Nginx) to serve the UI.
- ✓ Backend API (Python/Node) to handle requests.
- ✓ Database (PostgreSQL) to store data.
- ✓ Each component runs as a Deployment.
- ✓ Each component is exposed with a Service.
- ✓ Frontend communicates with backend.
- ✓ Backend communicates with database.

ARCHITECTURE OVERVIEW



1. FRONTEND DEPLOYMENT (Nginx)

frontend-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

2. BACKEND DEPLOYMENT (API)

backend-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: api
          image: hashicorp/http-echo
          args: ["-text=API"]
          ports:
            - containerPort: 5678
```

3. DATABASE DEPLOYMENT

db-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: database
spec:
  replicas: 1
  selector:
    matchLabels:
      app: database
  template:
    metadata:
      labels:
        app: database
    spec:
      containers:
        - name: postgres
          image: postgres:15
          env:
            - name: POSTGRES_PASSWORD
              value: veriqtal23
          ports:
            - containerPort: 5432
```

4. CREATE SERVICES

Example: frontend-service.yaml (NodePort)

```
apiVersion: v1
kind: Service
metadata:
  name: frontend-svc
spec:
  type: NodePort
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

TEST THE APPLICATION

- 1 Apply all manifests:
kubectl apply -f .
- 2 Check that everything is running:
kubectl get pods
kubectl get services
kubectl get deployments
- 3 Access the frontend:
http://<Node-IP>:30080
- 4 The frontend should be able to talk to the backend, and the backend should be able to connect to the database.

COMMON BEGINNER MISTAKES

- ✗ Using the wrong service name in the frontend.
- ✗ Hardcoding the database IP instead of using the service name.
- ✗ Pods cannot communicate due to wrong service selectors.
- ✗ Forgetting to expose the frontend service.
- ✗ Database not starting due to missing password or persistence.

PRACTICE THIS

- ✓ Change the number of replicas for frontend and backend.
- ✓ Update the backend image to a custom app.
- ✓ Try accessing the backend directly using its service DNS name inside a pod.



JUNIOR ENGINEER TIP

Inside the cluster, services talk to each other using the service name, not the Pod IP. For example: <http://backend-svc:5678>.



NEXT STEP

In the next project, you will move application configuration to ConfigMaps and Secrets.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Configure Applications Properly

Move application configuration into ConfigMaps and sensitive values into Secrets.
Update configuration without rebuilding the image.

WHAT YOU WILL LEARN

- ✓ What ConfigMaps and Secrets are.
- ✓ When to use ConfigMaps vs Secrets.
- ✓ Create and use them in a real application.
- ✓ Inject values as environment variables and as files.
- ✓ Update configuration without rebuilding the image.
- ✓ Verify that the application reads the new values.

HOW IT WORKS



1. CREATE A CONFIGMAP

File: `app-configmap.yaml`

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  labels:
    app: myapp
data:
  APP_ENV: production
  APP_DEBUG: "false"
  APP_MESSAGE: "Welcome to VERIQTA"
```

2. CREATE A SECRET

File: `app-secret.yaml`

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
  labels:
    app: myapp
type: Opaque
data:
  DB_PASSWORD: dmVyaXG0YTEyMw==
  API_KEY: YWJMTzZNDU2
```

✓ Values are base64 encoded.
Use: `echo -n "value" | base64`

3. USE THEM IN A DEPLOYMENT

File: `app-deployment.yaml` (excerpt)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  template:
    spec:
      containers:
        - name: myapp
          image: myapp:1.0
          env:
            - name: APP_ENV
              valueFrom:
                configMapKeyRef:
                  name: app-config
                  key: APP_ENV
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: app-secret
                  key: DB_PASSWORD
```

4. APPLY THE MANIFESTS

```
kubectl apply -f app-configmap.yaml
kubectl apply -f app-secret.yaml
kubectl apply -f app-deployment.yaml
```

- ✓ Check that the resources are created:


```
kubectl get configmaps
kubectl get secrets
kubectl get pods
```

5. UPDATE CONFIGURATION

- ✓ Edit the ConfigMap (change a value):
`kubectl edit configmap app-config`
- ✓ OR apply a new file:
`kubectl apply -f app-configmap.yaml`
- ✓ Restart the deployment to pick up the new values (for environment variables):
`kubectl rollout restart deployment myapp`
- ✓ Verify the new configuration:
`kubectl logs deployment/myapp`

CONFIGMAP vs SECRET

Feature	ConfigMap	Secret
Stores	Non-sensitive data	Sensitive data
Examples	App settings, feature flags	Passwords, API keys, tokens
Encoded	Plain text	Base64 encoded (not encrypted)
Use case	Application configuration	Sensitive information
Visible with	<code>kubectl get configmap -o yaml</code>	<code>kubectl get secret -o yaml</code>

COMMON BEGINNER MISTAKES

- ✗ Storing sensitive data in a ConfigMap.
- ✗ Committing real secrets to Git.
- ✗ Forgetting to restart the Pod after updating ConfigMap (for env vars).
- ✗ Using plain text secrets in production.
- ✗ Not using Secrets for database credentials.

JUNIOR ENGINEER TIP

Use ConfigMaps for configuration, Secrets for sensitive data, and consider external secret management (e.g. AWS Secrets Manager, HashiCorp Vault) for production environments.

PRACTICE THIS

- ✓ Add a new environment variable to the ConfigMap.
- ✓ Mount the ConfigMap as a file instead of environment variables.
- ✓ Update a value and verify the application reads the new value.
- ✓ Create a Secret for an API key and use it in the application.

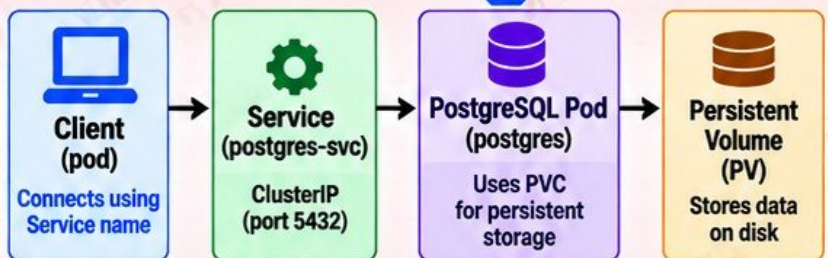
Persistent Database Deployment

Deploy PostgreSQL with persistent storage. Work with PersistentVolumes, PersistentVolumeClaims, StorageClasses, and data persistence after Pod recreation.

WHAT YOU WILL LEARN

- ✓ Deploy PostgreSQL in Kubernetes.
- ✓ Use PersistentVolumes (PV) and PersistentVolumeClaims (PVC).
- ✓ Understand StorageClasses.
- ✓ Store database data persistently.
- ✓ Verify data remains after Pod recreation.
- ✓ Access the database from inside the cluster.
- ✓ Understand common storage issues and how to fix them.

ARCHITECTURE OVERVIEW



1. CREATE A STORAGE CLASS

File: storageclass.yaml

```

apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: standard
provisioner: kubernetes.io/aws-efs
parameters:
  type: gp3
  fsType: ext4
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer
  
```

2. CREATE A PVC

File: postgres-pvc.yaml

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: standard
  
```

3. DEPLOY POSTGRESQL

File: postgres-deployment.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:15
          env:
            - name: POSTGRES_PASSWORD
              value: veriqta123
          ports:
            - containerPort: 5432
          volumeMounts:
            - name: postgres-storage
              mountPath: /var/lib/postgresql/data
      volumes:
        - name: postgres-storage
          persistentVolumeClaim:
            claimName: postgres-pvc
  
```

4. CREATE A SERVICE

File: postgres-service.yaml

```

apiVersion: v1
kind: Service
metadata:
  name: postgres-svc
spec:
  type: ClusterIP
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
  
```

5. APPLY AND VERIFY

```

kubectl apply -f storageclass.yaml
kubectl apply -f postgres-pvc.yaml
kubectl apply -f postgres-deployment.yaml
kubectl apply -f postgres-service.yaml
  
```

✓ Check resources:

```

kubectl get storageclass
kubectl get pvc
kubectl get pods
kubectl get svc
  
```

6. TEST THE DATABASE

Connect from a temporary pod:

```

kubectl run postgres-client --rm -it \
  --image=postgres:15 -- bash
psql -h postgres-svc -U postgres
Enter password: veriqta123
  
```

VERIFY DATA PERSISTENCE

- 1 Create a test table and insert data.
- 2 Delete the PostgreSQL pod:
`kubectl delete pod -l app=postgres`
- 3 Wait for the new pod to start.
- 4 Connect again and confirm the data still exists.

✓ The data persists because it is stored on the Persistent Volume, not in the pod.

COMMON BEGINNER MISTAKES

- ✗ Not using a PersistentVolumeClaim.
- ✗ Using emptyDir (data is lost).
- ✗ Wrong storageClassName.
- ✗ Expecting data to persist without a PVC.
- ✗ Deleting the PVC (this can delete the data).
- ✗ Using the wrong Service name to connect.
- ✗ Forgetting to set a secure password.

PRACTICE THIS

- ✓ Change the storage size to 20Gi.
- ✓ Use a different StorageClass (e.g. gp2, local).
- ✓ Create a second table and verify persistence.
- ✓ Scale the deployment (replicas: 2) and observe how the volume behaves.
- ✓ Try deleting the PVC (in a non-production cluster) and observe what happens.



JUNIOR ENGINEER TIP

Stateful data must be stored outside the pod. Pods are tempary. PersistentVolumes ensure your data survives pod failures, rescheduling, and recreations.



NEXT STEP

In the next project, you will configure resource requests, limits and rolling updates for a production-style application.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

ENGINEER TODAY
A HIGHER TOMORROW



LEARN
BUILD
SHARE
GROW

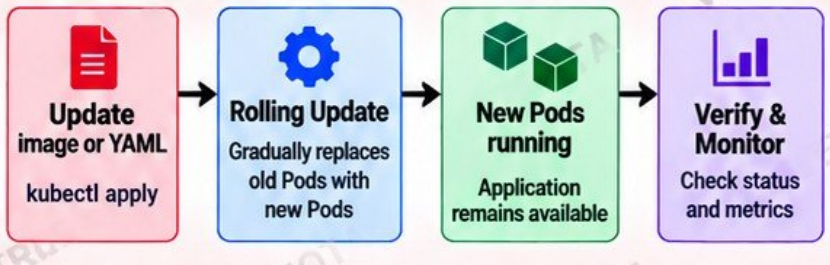
Production-Style Application Deployment

Add resource requests and limits, labels, annotations, rolling updates, revision history, and rollback.

WHAT YOU WILL LEARN

- ✓ Set resource requests and limits.
- ✓ Add useful labels and annotations.
- ✓ Configure a rolling update strategy.
- ✓ View deployment revision history.
- ✓ Rollback to a previous version.
- ✓ Understand why these settings matter in production.
- ✓ Verify the deployment and monitor the rollout.

DEPLOYMENT FLOW (ROLLING UPDATE)



1. DEPLOYMENT YAML (PRODUCTION-STYLE)

File: `nginx-deployment.yaml`

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-app
  labels:
    app: nginx-app
    env: production
  annotations:
    team: platform
    description: "Nginx web application"
spec:
  replicas: 3
  revisionHistoryLimit: 5
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  selector:
    matchLabels:
      app: nginx-app
  template:
    metadata:
      labels:
        app: nginx-app
        env: production
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
  
```

2. APPLY AND CHECK

```

kubectl apply -f nginx-deployment.yaml
kubectl get deployments
kubectl get pods
kubectl describe deployment nginx-app
  
```

- ✓ You should see 3 running Pods and the specified resource limits.

3. UPDATE THE APPLICATION

Update the image to a new version (e.g. `nginx:1.26`).

```

kubectl set image deployment/nginx-app \
  nginx=nginx:1.26
  
```

- ✓ Kubernetes performs a rolling update. Old Pods are replaced gradually.

4. VIEW REVISION HISTORY

```

kubectl rollout history deployment/nginx-app
kubectl rollout history deployment/nginx-app \
  --revision=1
  
```

- ✓ Shows the list of previous revisions and the changes made.

5. ROLLBACK IF NEEDED

```

kubectl rollout undo deployment/nginx-app
kubectl rollout undo deployment/nginx-app \
  --to-revision=1
  
```

- ✓ Reverts to a previous working version.

KEY CONCEPTS

-  **Resource Requests**
Guarantees minimum CPU and memory for the container.
-  **Resource Limits**
Sets the maximum CPU and memory the container can use.
-  **Labels**
Identify and group resources.
-  **Annotations**
Store additional metadata (e.g. team, description, links).
-  **Rolling Update**
Updates Pods with zero or minimal downtime.
-  **Revision History**
Keeps previous versions for rollback.
-  **Rollback**
Revert to a previous version if there is an issue.

COMMON BEGINNER MISTAKES

- ✗ Not setting resource requests and limits.
- ✗ Using latest image tag in production.
- ✗ Forgetting labels and annotations.
- ✗ Setting replicas to 1 in production.
- ✗ Not checking the rollout status.
- ✗ Not keeping revision history.
- ✗ Making changes directly in the cluster instead of version control.



JUNIOR ENGINEER TIP

Resource requests and limits prevent one application from consuming all cluster resources. Rolling updates and rollback reduce downtime and make deployments safer.

PRACTICE THIS

- ✓ Deploy the application with 3 replicas.
- ✓ Update to a different Nginx version and observe the rollout.
- ✓ View the revision history.
- ✓ Perform a rollback to the previous version.
- ✓ Change the resource limits and monitor the effect.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



ENGINEER TODAY
A HIGHER TOMORROW

LEARN
BUILD
SHARE
GROW

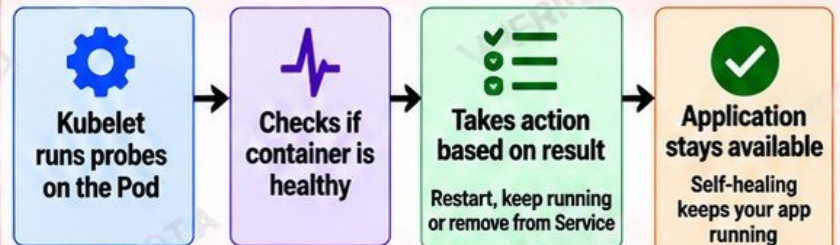
Application Health and Self-Healing

Configure startup, readiness, and liveness probes. Intentionally break health checks and investigate why Pods become unavailable or restart.

WHAT YOU WILL LEARN

- ✓ Understand the purpose of each probe.
- ✓ Configure startup, readiness, and liveness probes.
- ✓ See how Kubernetes uses probe results.
- ✓ Intentionally break health checks.
- ✓ Troubleshoot failing probes using logs and events.
- ✓ Understand how self-healing works.

HOW PROBES WORK



1. STARTUP PROBE

Used during application startup.

- ✓ Gives the application time to start.
- ✓ Prevents liveness probe from killing a slow-starting container.
- ✓ Used for applications that take longer to initialize (e.g. databases).

2. READINESS PROBE

Checks if the application is ready to serve traffic.

- ✓ If it fails, the Pod is removed from the Service.
- ✓ Traffic will not be sent to the Pod.
- ✓ Used when the app is starting or temporarily unavailable (e.g. during a deploy).

3. LIVENESS PROBE

Checks if the application is still running properly.

- ✓ If it fails, Kubernetes restarts the container.
- ✓ Used to recover from application failures (e.g. deadlocks, memory leaks).
- ✓ Do not use liveness probe for slow-starting applications (use startup probe).

4. DEPLOYMENT YAML WITH PROBES

File: app-deployment.yaml (excerpt)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: myapp
          image: nginx:1.25
          ports:
            - containerPort: 80
          startupProbe:
            httpGet:
              path: /health
              port: 80
              initialDelaySeconds: 10
              periodSeconds: 10
              failureThreshold: 30
          readinessProbe:
            httpGet:
              path: /health
              port: 80
              initialDelaySeconds: 5
              periodSeconds: 10
              failureThreshold: 3
          livenessProbe:
            httpGet:
              path: /health
              port: 80
              initialDelaySeconds: 15
              periodSeconds: 10
              failureThreshold: 3
  
```

5. APPLY AND CHECK

```

kubectl apply -f app-deployment.yaml
kubectl get pods
kubectl describe pod <pod-name>
  
```

- ✓ Check the Events section to see probe results.

6. INTENTIONALLY BREAK A PROBE

Example: Change the health endpoint to a wrong path.

```

kubectl edit deployment myapp
Change the path from /health to /wrong
kubectl apply -f app-deployment.yaml
kubectl get pods
  
```

- ! Observe how the Pod becomes Unready or restarts.

7. TROUBLESHOOTING COMMANDS

```

kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl get events --sort-by=.lastTimestamp
  
```

- ✓ Look for probe failure messages, reason and timestamps.

POD STATES EXAMPLE

Running	NotReady	CrashLoopBackOff
All probes passing. Pod is healthy.	Readiness probe failing. Removed from Service.	Liveness probe failing. Container restarts repeatedly.

COMMON BEGINNER MISTAKES

- ✗ Using liveness probe for slow-starting apps.
- ✗ Wrong path or port in the probe.
- ✗ Setting very short initialDelaySeconds.
- ✗ Expecting traffic to go to a Pod that is NotReady.
- ✗ Ignoring Events and logs.
- ✗ Not testing probe failures.

PRACTICE THIS

- ✓ Deploy the application with all three probes.
- ✓ Change the probe path to break the readiness probe.
- ✓ Change the probe to a failing command to break liveness probe.
- ✓ Observe the Pod restart and check the logs.
- ✓ Fix the configuration and verify the application is healthy again.

JUNIOR ENGINEER TIP

Readiness keeps bad Pods out of the Service.
Liveness brings unhealthy Pods back to life.
Startup gives your application time to initialize.

NEXT STEP

In the next project, you will expose the application with an Ingress and configure HTTPS with TLS.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



LEARN
BUILD
SHARE
GROW

Deploy an Ingress Controller and route HTTP traffic by hostname or path. Understand Service vs Ingress and the external request path.

**ENGINEER TODAY
A HIGHER TOMORROW**

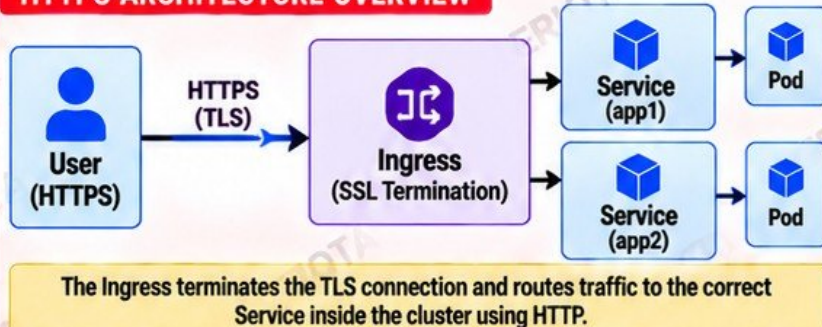
Deploy HTTPS with TLS

Add TLS to the application. Work with certificates, Kubernetes TLS Secrets, HTTPS termination, and certificate management concepts.

WHAT YOU WILL LEARN

- ✓ Why HTTPS and TLS are important.
- ✓ How TLS works in Kubernetes.
- ✓ Create and use Kubernetes TLS Secrets.
- ✓ Configure HTTPS in an Ingress.
- ✓ Understand TLS termination.
- ✓ Use cert-manager for automatic certificates (Let's Encrypt).
- ✓ Verify and troubleshoot TLS.

HTTPS ARCHITECTURE OVERVIEW



1. GENERATE A TLS CERTIFICATE

For testing, create a self-signed certificate (or use a real certificate later).

```
openssl req -x509 -nodes -days 365 \
  -newkey rsa:2048 \
  -keyout tls.key \
  -out tls.crt \
  -subj "/CN=app1.veriqta.local"
```

- ✓ This creates a private key (tls.key) and a certificate (tls.crt).

4. VERIFY HTTPS ACCESS

Test the application using HTTPS.

```
curl -k https://app1.veriqta.local
```

- ✓ You should see the application response.
- ✓ Use -k for self-signed certs (testing only).
- ✓ For a trusted certificate, you don't need -k.

2. CREATE A KUBERNETES TLS SECRET

Create a TLS secret using the certificate files from step 1.

```
kubectl create secret tls app-tls \
  --cert=tls.crt \
  --key=tls.key
```

- ✓ This secret will be used by the Ingress to terminate TLS.

5. AUTOMATE CERTIFICATES WITH CERT-MANAGER

Use cert-manager with Let's Encrypt (for production).

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: your-email@veriqta.com
    privateKeySecretRef:
      name: letsencrypt-prod
    solvers:
      - http01:
          ingress:
            class: nginx
```

3. CONFIGURE INGRESS WITH TLS

Add TLS configuration to the Ingress.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
spec:
  tls:
    - hosts:
        - app1.veriqta.local
      secretName: app-tls
  rules:
    - host: app1.veriqta.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: app1-svc
                port:
                  number: 80
```

- ✓ The Ingress uses the TLS secret for HTTPS connections.

6. TLS TERMINATION OPTIONS

Option	Where TLS is Terminated	Use Case
Ingress (recommended)	At the Ingress Controller	Most common for web apps
Service (rare)	At the Service/Pod	End-to-end encryption
External Load Balancer	At cloud load balancer (e.g. AWS ALB, GCP LB)	Cloud-managed TLS
Pod (application)	Inside the application	Custom requirements

COMMON BEGINNER MISTAKES

- ✗ Using an invalid or expired certificate.
- ✗ Wrong hostname in the certificate.
- ✗ Not setting the correct secretName.
- ✗ Forgetting to update DNS / hosts file.
- ✗ Using self-signed certificates in production.
- ✗ Not verifying the certificate chain.
- ✗ Mixed content (HTTP and HTTPS).

PRACTICE THIS

- ✓ Deploy HTTPS for your application.
- ✓ Use a real domain or local /etc/hosts.
- ✓ Try both self-signed and Let's Encrypt.
- ✓ Inspect the certificate using: `openssl s_client -connect app1.veriqta.local:443`
- ✓ Experiment with different TLS termination options.
- ✓ Intentionally use an expired cert and observe the error.



JUNIOR ENGINEER TIP

In production, use cert-manager with Let's Encrypt or your organization's CA. Always monitor certificate expiry and set up automatic renewal.



NEXT STEP

In the next project, you will build a highly available application with multiple replicas and test what happens when a Pod fails.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

LEARN
BUILD
SHARE
GROW



ENGINEER TODAY
A HIGHER TOMORROW

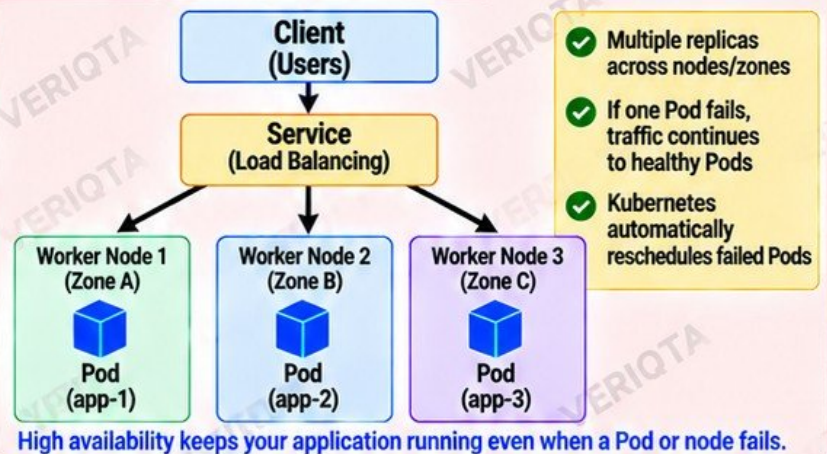
Build a Highly Available Application

Run multiple replicas, use Pod anti-affinity or topology spread constraints, and test what happens when a Pod disappears.

WHAT YOU WILL LEARN

- ✓ Why high availability matters in Kubernetes.
- ✓ Run multiple replicas of an application.
- ✓ Use Pod anti-affinity or topology spread constraints to improve resilience.
- ✓ Understand how Kubernetes reschedules Pods when one fails.
- ✓ Test what happens when a Pod disappears.
- ✓ Verify that the application remains available.
- ✓ Understand real-world HA considerations and limitations.

HIGH AVAILABILITY ARCHITECTURE



1. DEPLOY MULTIPLE REPLICAS

Create a Deployment with multiple replicas.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ha-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: ha-app
  template:
    metadata:
      labels:
        app: ha-app
    spec:
      containers:
        - name: app
          image: nginx:1.25
          ports:
            - containerPort: 80
```

- ✓ Kubernetes will distribute the replicas across available nodes.

2. USE POD ANTI-AFFINITY

Avoid running multiple Pods on the same node.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ha-app
spec:
  template:
    spec:
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchLabels:
                  app: ha-app
              topologyKey: kubernetes.io/hostname
```

- ✓ This ensures replicas are not scheduled on the same node, improving resilience.

3. USE TOPOLOGY SPREAD CONSTRAINTS

Spread Pods across zones or nodes.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ha-app
spec:
  template:
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: DoNotSchedule
          labelSelector:
            matchLabels:
              app: ha-app
```

- ✓ This spreads replicas evenly across zones or nodes to reduce the impact of a single node failure.

4. SIMULATE A POD FAILURE

Delete a Pod and observe what happens.

```
kubectl get pods
kubectl delete pod <pod-name>
kubectl get pods -w
```

- ✓ Kubernetes will automatically create a new Pod to maintain the desired number of replicas.

5. VERIFY HIGH AVAILABILITY

Check that the application remains available.

```
kubectl get pods -o wide
kubectl get nodes
kubectl get svc
curl http://<service-ip>
```

- ✓ You should see the application still running and accessible.

6. MONITOR AND TEST FURTHER

- ✓ Check which nodes the Pods are running on.
- ✓ Simulate a node failure (cordon and drain).
- ✓ Observe how Pods are rescheduled.
- ✓ Test with higher load.
- ✓ Verify no downtime for your application.

JUNIOR ENGINEER TIP



High availability is not just about replicas, it is about smart placement, proper configuration, and testing failure scenarios.

COMMON BEGINNER MISTAKES

- ✗ Using only 1 replica.
- ✗ Not using anti-affinity or topology spread.
- ✗ Testing only in a single node cluster.
- ✗ Assuming replicas alone guarantee HA.
- ✗ Not verifying rescheduling after failures.

NEXT STEP



In the next project, you will configure monitoring and alerting for your application.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



ENGINEER TODAY
A HIGHER TOMORROW

LEARN
BUILD
SHARE
GROW

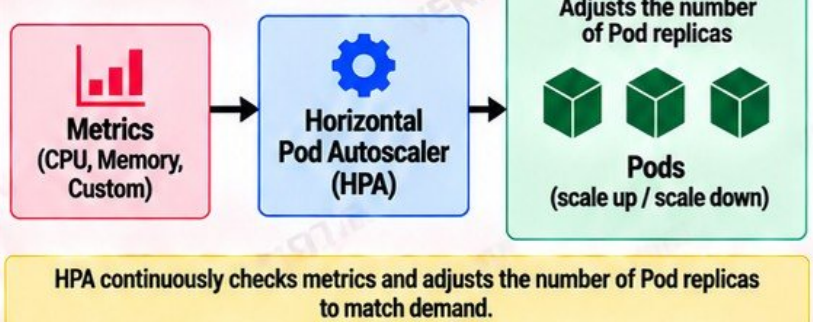
Autoscale an Application

Configure the Horizontal Pod Autoscaler, generate load, observe scaling, and understand metrics, requests, thresholds, and scaling limitations.

WHAT YOU WILL LEARN

- ✓ What the Horizontal Pod Autoscaler (HPA) is and how it works.
- ✓ How metrics, requests and thresholds affect scaling.
- ✓ Deploy an application with resource requests.
- ✓ Configure an HPA using CPU (and optional custom metrics).
- ✓ Generate load and observe scaling in real time.
- ✓ Understand scale up and scale down behavior.
- ✓ Learn the limitations and best practices for autoscaling.

HOW HPA WORKS



1. DEPLOY AN APPLICATION

Deploy a simple application with resource requests.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: hpa-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: hpa-app
  template:
    metadata:
      labels:
        app: hpa-app
    spec:
      containers:
        - name: app
          image: nginx:latest
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
  
```

- ✓ Resource requests are required for CPU-based autoscaling.

2. CREATE A HORIZONTAL POD AUTOSCALER

Create an HPA to scale based on CPU utilization.

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: hpa-app
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: hpa-app
  minReplicas: 1
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
  
```

- ✓ This will keep the average CPU utilization around 70% by scaling between 1 and 10 replicas.

3. GENERATE LOAD AND OBSERVE SCALING

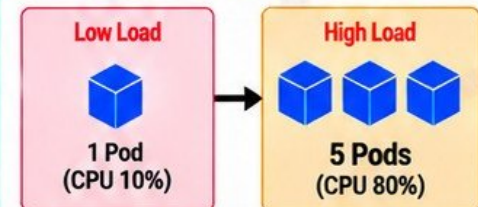
Generate load and watch the replicas increase.

```

# Generate load (using hey)
hey -n 10000 -c 100 http://<service-ip>

# Watch the HPA
kubectl get hpa hpa-app -w

# Watch the pods
kubectl get pods -l app=hp-app -w
  
```



- ✓ As load increases, the HPA automatically adds more replicas. When the load decreases, it scales back down.

4. USE DIFFERENT METRICS (OPTIONAL)

You can also use memory or custom metrics.

```

metrics:
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 70
  
```

- ✓ HPA can use CPU, memory or custom metrics (e.g. from Prometheus).

5. UNDERSTAND SCALING BEHAVIOR

- ✓ HPA checks metrics every 15 seconds (by default).
- ✓ Scale up happens quickly.
- ✓ Scale down is slower (default 5 minutes).
- ✓ Scaling is based on average metrics across Pods.
- ✓ Pods must have resource requests defined.
- ✓ Not suitable for stateful workloads without proper design.

6. COMMON LIMITATIONS

- ✗ HPA cannot scale to zero (use KEDA for that).
- ✗ Needs metrics server or custom metrics.
- ✗ Scaling is based on averages (may hide spikes).
- ✗ Not ideal for workloads with long startup times.
- ✗ Does not handle application-level scaling logic.
- ✗ Requires proper resource requests.
- ✗ Cannot predict future load (reacts to current metrics).

PRACTICE THIS

- ✓ Deploy the application and configure the HPA.
- ✓ Generate CPU load and observe scaling.
- ✓ Try memory-based autoscaling.
- ✓ Experiment with different thresholds.
- ✓ Test scale down behavior.
- ✓ Explore custom metrics (optional).

JUNIOR ENGINEER TIP

Autoscaling keeps your application responsive and cost-efficient. Always set realistic resource requests and monitor your application to avoid over- or under-scaling.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



LEARN
BUILD
SHARE
GROW

ENGINEER TODAY
A HIGHER TOMORROW

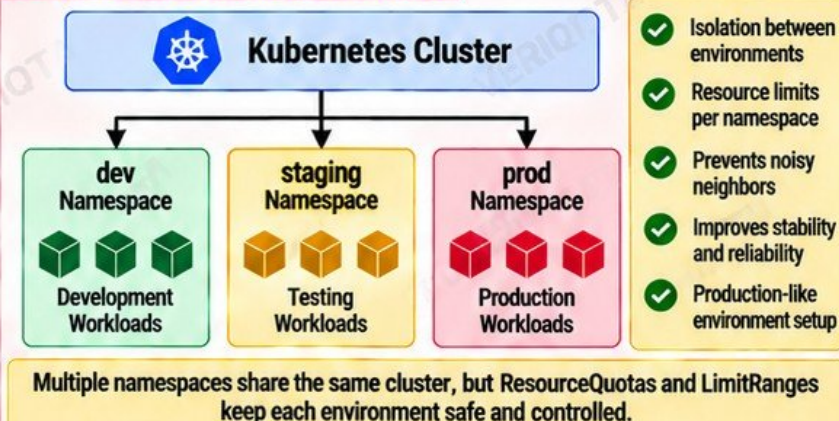
Organize a Shared Cluster

Build dev, staging, and production-style namespaces. Add ResourceQuotas and LimitRanges to prevent one workload from consuming the cluster.

WHAT YOU WILL LEARN

- ✓ Why namespaces are important.
- ✓ How to create and organize dev, staging, and production namespaces.
- ✓ Use ResourceQuotas to limit total usage per namespace.
- ✓ Use LimitRanges to set default and maximum resource limits.
- ✓ Prevent one workload from consuming the entire cluster.
- ✓ Learn best practices for multi-tenant cluster environments.

SHARED CLUSTER ARCHITECTURE



1. CREATE NAMESPACES

Create dev, staging, and production namespaces.

```
kubectl create namespace dev
kubectl create namespace staging
kubectl create namespace prod
kubectl get namespaces
```

- ✓ Namespaces provide logical isolation for different environments.

4. DEPLOY APPLICATIONS TO NAMESPACES

Deploy the same application to different environments.

```
kubectl create deployment app \
--image=nginx --replicas=2 \
-n dev
kubectl create deployment app \
--image=nginx --replicas=2 \
-n staging
```

- ✓ Each environment runs independently in its own namespace.

6. TEST THE LIMITS

Try to create a workload that exceeds the limits.

```
kubectl run test --image=nginx \
--requests=cpu=10 --limits=cpu=20 -n dev
```

Error from server (Forbidden):
exceeded quota: dev-quota, requested: cpu=10, used: 4, limited: 8

- ✓ Kubernetes will block workloads that exceed the ResourceQuota or LimitRange.

2. SET RESOURCE QUOTAS

Limit total resource usage in a namespace.

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-quota
  namespace: dev
spec:
  hard:
    requests.cpu: "4"
    requests.memory: "8Gi"
    limits.cpu: "8"
    limits.memory: "16Gi"
```

- ✓ ResourceQuotas prevent a single namespace from consuming all cluster resources.

5. VERIFY QUOTAS AND LIMITS

Check the configured quotas and limits.

```
kubectl get resourcequota -n dev
kubectl describe resourcequota dev-quota
kubectl get limitrange -n dev
kubectl describe limitrange dev-limits
```

- ✓ Always verify that quotas and limits are applied correctly.

COMMON BEGINNER MISTAKES

- ✗ Not using namespaces for environment isolation.
- ✗ Setting quotas too low or too high.
- ✗ Forgetting to set both requests and limits.
- ✗ Not verifying quotas and limits.
- ✗ Assuming namespaces provide security (use RBAC).
- ✗ Letting one namespace consume all resources.

3. SET LIMIT RANGES

Define default and maximum resource limits for containers in a namespace.

```
apiVersion: v1
kind: LimitRange
metadata:
  name: dev-limits
  namespace: dev
spec:
  limits:
    - type: Container
      default:
        cpu: "500m"
        memory: "512Mi"
      defaultRequest:
        cpu: "100m"
        memory: "128Mi"
      max:
        cpu: "2"
        memory: "4Gi"
```

- ✓ LimitRanges ensure workloads have reasonable default and maximum resource values.

PRACTICE THIS

- ✓ Create dev, staging, and prod namespaces.
- ✓ Set ResourceQuotas and LimitRanges for each.
- ✓ Deploy applications to each namespace.
- ✓ Try to exceed the quotas and observe the error.
- ✓ Monitor resource usage with `kubectl top`.
- ✓ Explore how namespaces, quotas, and limits work together.

JUNIOR ENGINEER TIP

A shared cluster is like a shared apartment. Namespaces give each team a room, and ResourceQuotas and LimitRanges set boundaries so no one takes over the whole building.

NEXT STEP

In the next project, you will implement Role-Based Access Control (RBAC) to manage permissions.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



LEARN
BUILD
SHARE
GROW

ENGINEER TODAY
A HIGHER TOMORROW

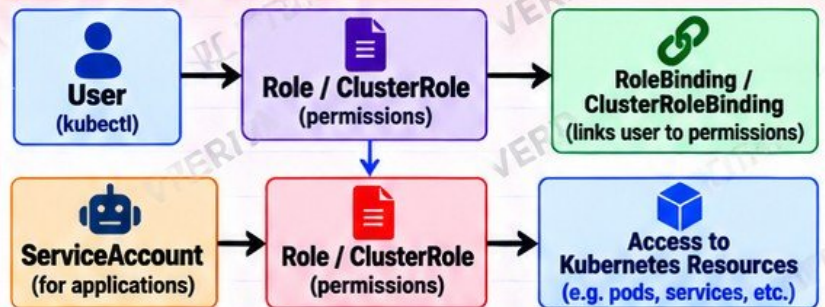
Secure Access with RBAC

Create ServiceAccounts, Roles, RoleBindings, ClusterRoles, and ClusterRoleBindings.
Test least-privilege access with `kubectl auth can-i`.

WHAT YOU WILL BUILD

- Create a ServiceAccount for an application.
- Create a Role with limited permissions.
- Bind the Role to the ServiceAccount.
- Create a ClusterRole for cluster-wide access.
- Understand the difference between Role and ClusterRole.
- Test permissions using `kubectl auth can-i`.
- Apply the principle of least privilege.

RBAC ARCHITECTURE (CONCEPT)



1. CREATE A NAMESPACE

Work in a dedicated namespace.

```
kubectl create namespace rbac-demo
kubectl get namespaces
```

2. CREATE A SERVICEACCOUNT

ServiceAccounts are used by applications running in Pods.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  namespace: rbac-demo
```

3. CREATE A ROLE (NAMESPACE SCOPE)

This role allows read access to Pods.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: rbac-demo
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

4. BIND THE ROLE TO THE SERVICEACCOUNT

This gives the ServiceAccount the permissions from the Role.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: app-sa-binding
  namespace: rbac-demo
subjects:
- kind: ServiceAccount
  name: app-sa
  namespace: rbac-demo
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

5. CREATE A CLUSTERROLE (CLUSTER SCOPE)

This role allows read access to nodes (cluster-wide).

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["get", "list", "watch"]
```

6. BIND THE CLUSTERROLE

This gives the user or ServiceAccount cluster-wide access.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: node-reader-binding
subjects:
- kind: ServiceAccount
  name: app-sa
  namespace: rbac-demo
roleRef:
  kind: ClusterRole
  name: node-reader
  apiGroup: rbac.authorization.k8s.io
```

7. TEST PERMISSIONS

Verify what the ServiceAccount can do.

```
# Can it list pods?
kubectl auth can-i list pods \
--as=system:serviceaccount:rbac-demo:app-sa

# Can it get nodes? (should be no)
kubectl auth can-i get nodes \
--as=system:serviceaccount:rbac-demo:app-sa
```

COMMON BEGINNER MISTAKES

- ✗ Giving cluster-admin when not needed.
- ✗ Using ClusterRole for namespaced resources.
- ✗ Binding the wrong ServiceAccount or namespace.
- ✗ Forgetting to specify the namespace.
- ✗ Not testing permissions with `kubectl auth can-i`.

PRACTICE THIS

- ✓ Create a new Role that allows access to Services and ConfigMaps.
- ✓ Create a ServiceAccount for a database application with minimal permissions.
- ✓ Test different permissions using `kubectl auth can-i`.
- ✓ Try creating a Role for write access and compare the risk.



JUNIOR ENGINEER TIP

Always follow the principle of least privilege. Give only the permissions that are required, nothing more.



NEXT STEP

In the next project, you will control Pod-to-Pod traffic using NetworkPolicies.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



LEARN
BUILD
SHARE
GROW

ENGINEER TODAY
A HIGHER TOMORROW

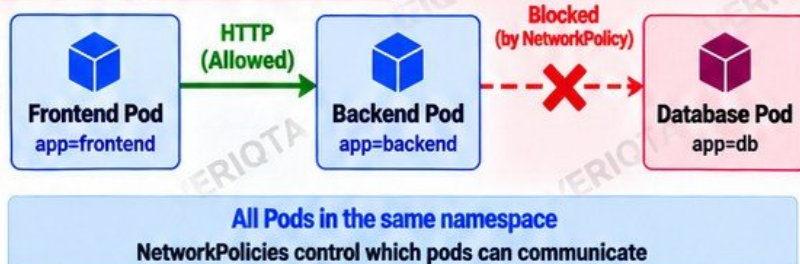
Control Pod-to-Pod Traffic

Introduce NetworkPolicies. Start with allowed communication, apply restrictions, deliberately break connectivity, then diagnose the failure.

WHAT YOU WILL BUILD

- ✓ Deploy two applications in the same namespace.
- ✓ Allow communication between them.
- ✓ Apply a NetworkPolicy to restrict traffic.
- ✓ Test and observe the blocked connection.
- ✓ Diagnose the failure using real commands.
- ✓ Understand how NetworkPolicies control pod-to-pod traffic.
- ✓ Learn to use labels and selectors.
- ✓ Apply the principle of least privilege.

ARCHITECTURE OVERVIEW



1. DEPLOY TEST APPLICATIONS

Deploy a frontend, backend and database.

```
kubectl create namespace np-demo
kubectl apply -f frontend.yaml
kubectl apply -f backend.yaml
kubectl apply -f database.yaml
```

```
kubectl get pods -n np-demo
kubectl get services -n np-demo
```

5. DIAGNOSE THE FAILURE

When the connection fails, check the following:

```
kubectl get networkpolicies -n np-demo
kubectl describe networkpolicy \
  deny-all-backend -n np-demo
kubectl get pods -n np-demo --show-labels
kubectl exec -n np-demo deploy/frontend --
  curl -v http://backend:80
```



COMMON ERROR

curl: (28) Connection timed out
This means the NetworkPolicy is blocking the traffic.

2. ALLOW FRONTEND TO BACKEND

Create a NetworkPolicy to allow only frontend pods to access backend pods.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: np-demo
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - protocol: TCP
      port: 80
```

3. TEST THE CONNECTION

Test connectivity from frontend to backend.

```
kubectl exec -n np-demo \
  deploy/frontend -- curl \
  http://backend:80
```



EXPECTED RESULT

You should see a successful response from the backend application.

4. BLOCK ALL TRAFFIC

Now deny all access to the backend.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all-backend
  namespace: np-demo
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
```



EXPECTED RESULT

The connection from frontend to backend will now fail (connection timeout).

COMMON BEGINNER MISTAKES

- ✗ Forgetting to set pod labels.
- ✗ Applying the policy to the wrong namespace.
- ✗ Expecting NetworkPolicy to work without a CNI that supports it.
- ✗ Not checking the pod labels and selectors.
- ✗ Assuming traffic is allowed by default.

KEY CONCEPTS

- NetworkPolicy** - Controls pod-to-pod traffic.
- Ingress** - Defines incoming traffic rules.
- Egress** - Defines outgoing traffic rules.
- Pod Selector** - Selects pods using labels.
- CNI** - Container Network Interface (must support NetworkPolicy).

PRACTICE THIS

- ✓ Allow only frontend to backend.
- ✓ Block all traffic and verify the failure.
- ✓ Allow database access only from backend.
- ✓ Create an egress rule to allow only DNS.
- ✓ Try different label combinations.
- ✓ Remove the policy and confirm traffic works.



JUNIOR ENGINEER TIP

NetworkPolicies are deny-by-default for the selected pods. Always test from inside the cluster to confirm the rules.



NEXT STEP

In the next project, you will package your application with Helm for easier deployment and management.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



LEARN
BUILD
SHARE
GROW

ENGINEER TODAY
A HIGHER TOMORROW

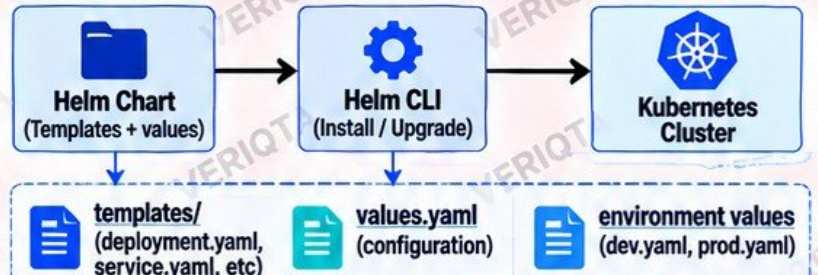
Package the Application with Helm

Convert repeated Kubernetes manifests into a Helm chart. Create templates, values.yaml, environment-specific values, upgrades, and rollbacks.

WHAT YOU WILL BUILD

- ✓ Turn Kubernetes manifests into a Helm chart.
- ✓ Create reusable templates.
- ✓ Use values.yaml for configuration.
- ✓ Create environment-specific values (dev, staging, prod).
- ✓ Deploy, upgrade and rollback the application.
- ✓ Understand Helm chart structure and best practices.

HOW HELM WORKS



1. CREATE A HELM CHART

Create a new chart using Helm.

```
helm create nginx-app
cd nginx-app
ls -R
```

CHART STRUCTURE (IMPORTANT FILES)

```

nginx-app/
├── Chart.yaml (chart metadata)
├── values.yaml (default values)
├── templates/ (Kubernetes manifests)
│   ├── deployment.yaml
│   ├── service.yaml
│   └── _helpers.tpl
└── charts/ (dependencies)
  
```

2. USE VALUES.YAML

Configure your application in values.yaml.

```

replicaCount: 2
image:
  repository: nginx
  tag: "1.25"
service:
  type: ClusterIP
  port: 80
  
```

ENVIRONMENT-SPECIFIC VALUES

Create separate files for each environment.

```

values.yaml           # default
values-dev.yaml       # development
values-staging.yaml   # staging
values-prod.yaml      # production
  
```

3. DEPLOY WITH HELM

Install the chart using your values file.

```

helm install nginx-app .
helm install nginx-app -f values-dev.yaml
helm list
  
```

4. UPGRADE THE APPLICATION

Change values and upgrade.

```

helm upgrade nginx-app -f values-prod.yaml
helm history nginx-app
  
```

5. ROLLBACK IF NEEDED

Rollback to a previous version.

```

helm rollback nginx-app 1
helm history nginx-app
  
```

KEY HELM CONCEPTS

Chart - A package of Kubernetes manifests.
templates - Files with placeholders.
values.yaml - Configuration values.
Release - A running instance of a chart.
Helm Repository - Stores and shares charts.
Upgrade - Apply changes to a release.
Rollback - Revert to a previous version.

COMMON BEGINNER MISTAKES

- ✗ Hardcoding values in templates.
- ✗ Forgetting to use values.yaml.
- ✗ Not testing the chart before deploying.
- ✗ Overriding values in the wrong file.
- ✗ Not checking helm history.
- ✗ Ignoring dependencies.
- ✗ Making breaking changes without a plan.

PRACTICE THIS

- ✓ Create a Helm chart for your application.
- ✓ Move Deployment, Service and Ingress into templates.
- ✓ Use values-dev.yaml and values-prod.yaml.
- ✓ Deploy to a namespace.
- ✓ Upgrade the image version.
- ✓ Change the replica count using --set.
- ✓ Rollback to the previous version.



JUNIOR ENGINEER TIP

Use Helm to standardize deployments, reduce duplication and manage different environments safely.



NEXT STEP

In the next project, you will build a Kubernetes CI/CD pipeline to deploy your Helm chart automatically.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



LEARN
BUILD
SHARE
GROW

ENGINEER TODAY
A HIGHER TOMORROW

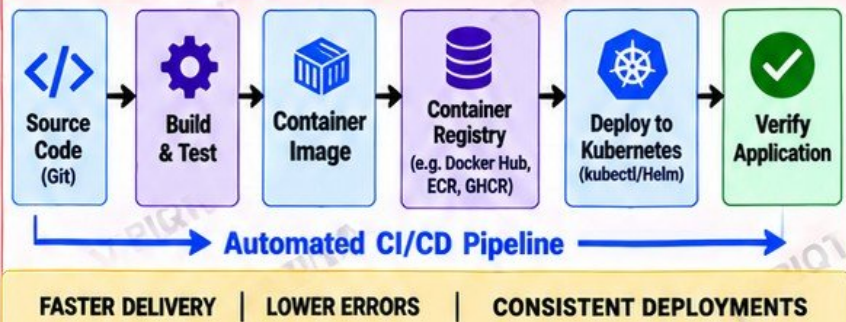
Build Kubernetes CI/CD

Source code → build → test → container image → registry → Kubernetes deployment → verification.
Teach why deployment credentials and rollback controls matter.

WHAT YOU WILL BUILD

- ✓ Build a complete CI/CD pipeline for a containerized application.
- ✓ Run automated tests.
- ✓ Build and push a Docker image to a container registry.
- ✓ Deploy to Kubernetes using kubectl or Helm.
- ✓ Use secure credentials (no hardcoded secrets).
- ✓ Implement rollback and verify the deployment.
- ✓ Understand how CI/CD reduces manual errors and speeds up delivery.

CI/CD PIPELINE OVERVIEW



1. APPLICATION CODE (GIT)

Store your application in a Git repository.

```
git clone https://github.com/your-org/app.git
cd app
```

2. BUILD AND TEST

Build the application and run tests.

```
# Example for a Node.js app
npm install
npm test

# Build Docker image
docker build -t myapp:1.0.0 .
```

3. PUSH TO CONTAINER REGISTRY

Tag and push the image to a registry.

```
docker tag myapp:1.0.0 your-username/
myapp:1.0.0
docker push your-username/myapp:1.0.0
```

4. DEPLOY TO KUBERNETES

Deploy using a manifest or Helm.

```
# Using kubectl
kubectl apply -f k8s/

# Using Helm
helm upgrade --install myapp ./chart \
--namespace production
```

5. VERIFY THE DEPLOYMENT

Check that the application is running.

```
kubectl get pods -n production
kubectl get services -n production
kubectl rollout status deployment/myapp \
-n production
```

6. ROLLBACK IF NEEDED

Rollback to the previous version.

```
kubectl rollout undo deployment/myapp \
-n production

# or with Helm
helm rollback myapp 1 -n production
```

DEPLOYMENT CREDENTIALS

- ✓ Use Kubernetes ServiceAccounts.
- ✓ Use minimal RBAC permissions.
- ✓ Store secrets in a secret manager (e.g. GitHub Secrets, AWS Secrets Manager).
- ✓ Never hardcode credentials in your code or pipeline.
- ✓ Rotate credentials regularly.

COMMON BEGINNER MISTAKES

- ✗ Hardcoding registry or cluster credentials.
- ✗ Skipping automated tests.
- ✗ Using the latest tag instead of a version tag.
- ✗ Giving too much cluster access to the CI/CD tool.
- ✗ Not verifying the deployment.
- ✗ No rollback plan.
- ✗ Pushing images without scanning.

PRACTICE THIS

- ✓ Set up a CI/CD pipeline (GitHub Actions, GitLab CI or Jenkins).
- ✓ Build, test and push a Docker image.
- ✓ Deploy to a Kubernetes namespace.
- ✓ Verify the application is running.
- ✓ Make a change and deploy a new version.
- ✓ Rollback to the previous version.
- ✓ Delete the deployment and redeploy using your pipeline.



JUNIOR ENGINEER TIP

CI/CD is not just about automation. It gives you repeatable, safer and faster deployments with easy rollback.



NEXT STEP

In the next project, you will implement GitOps with Argo CD to deploy your application automatically from Git.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

ENGINEER TODAY
A HIGHER TOMORROW



LEARN
BUILD
SHARE
GROW

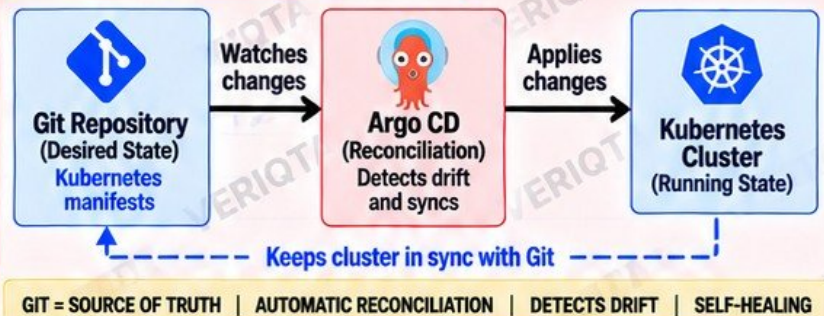
Implement GitOps Deployment

Store desired cluster state in Git and introduce reconciliation using Argo CD. Change Git, synchronize the cluster, detect drift, and recover desired state.

WHAT YOU WILL BUILD

- ✓ Deploy Argo CD in your Kubernetes cluster.
- ✓ Store Kubernetes manifests in a Git repository.
- ✓ Connect Argo CD to your Git repository.
- ✓ Automatically synchronize the cluster state.
- ✓ Change a manifest in Git and observe the update.
- ✓ Detect and fix configuration drift.
- ✓ Understand how GitOps improves security, consistency and reliability.

GITOPS ARCHITECTURE OVERVIEW



1. DEPLOY ARGO CD

Install Argo CD in your cluster.

```
kubectl create namespace argocd
kubectl apply -n argocd -f
https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

Get the Argo CD UI password:

```
kubectl -n argocd get secret
argocd-initial-admin-secret -o
jsonpath="{.data.password}" | base64 -d
```

Access the UI (port-forward):

```
kubectl -n argocd port-forward
svc/argocd-server 8080:443
```

2. CONNECT GIT REPOSITORY

Add your Git repository in Argo CD.

```
# Via CLI
argocd repo add
https://github.com/your-org/
k8s-manifests.git
---username <your-username>
---password <your-token>
```

USE A PERSONAL ACCESS TOKEN

Use a read-only token with minimal permissions to your repository.

3. CREATE AN APPLICATION

Tell Argo CD what to deploy.

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: myapp
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/
your-org/k8s-manifests.git
    targetRevision: main
    path: apps/myapp
  destination:
    server: https://kubernetes.default.svc
    namespace: myapp
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

4. MAKE A CHANGE IN GIT

Update the deployment image in Git.

```
spec:
  template:
    spec:
      containers:
        - name: app
          image: nginx:1.26 # changed
```

5. OBSERVE THE SYNCHRONIZATION

Argo CD detects the change and syncs it.

- ✓ See the change in Argo CD UI.
- ✓ Check application status is Synced.
- ✓ Verify new pods are running in Kubernetes.

```
kubectl get pods -n myapp
kubectl describe deployment myapp -n myapp
```

6. TEST DRIFT AND SELF-HEALING

Manually modify the cluster and see it revert.

```
# Change image directly (creates drift)
kubectl set image deployment/myapp
app=nginx:1.25 -n myapp
```

- ✓ Argo CD will detect the drift.
- ✓ It will automatically revert to the Git state (self-heal).
- ✓ Check the application events in Argo CD.

KEY CONCEPTS

- GitOps** - Infrastructure and apps managed in Git.
- Argo CD** - Continuously syncs desired state.
- Drift** - When cluster state differs from Git.
- Self-healing** - Automatically reverts changes.
- Sync policy** - Automated or manual sync.
- Prune** - Removes resources no longer in Git.

COMMON BEGINNER MISTAKES

- ✗ Using a token with too many permissions.
- ✗ Pointing to the wrong repository or path.
- ✗ Forgetting to set the correct namespace.
- ✗ Not understanding sync policy options.
- ✗ Manually changing resources in the cluster.
- ✗ Not checking application events/logs.

PRACTICE THIS

- ✓ Deploy Argo CD in your cluster.
- ✓ Connect a Git repository.
- ✓ Deploy a sample application.
- ✓ Update the image in Git and watch it sync.
- ✓ Create a deliberate drift and observe self-healing.
- ✓ Try manual vs automated sync policy.



JUNIOR ENGINEER TIP

With GitOps, you don't log in to the cluster to make changes. You change Git, and let the system do the work.



NEXT STEP

In the next project, you will build Kubernetes observability with Prometheus and Grafana.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



LEARN
BUILD
SHARE
GROW

ENGINEER TODAY
A HIGHER TOMORROW

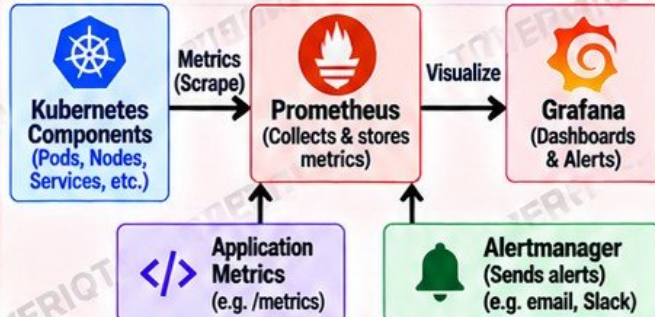
Build Kubernetes Observability

Deploy Prometheus and Grafana concepts around the application. Monitor Pods, CPU, memory, availability, application metrics, and investigate abnormal behavior.

WHAT YOU WILL BUILD

- ✓ Deploy Prometheus and Grafana in the cluster.
- ✓ Collect metrics from Kubernetes and your application.
- ✓ Monitor Pods, CPU, memory, and availability.
- ✓ Visualize metrics using Grafana dashboards.
- ✓ Set up alerts for abnormal behavior.
- ✓ Investigate real issues using metrics and logs.
- ✓ Understand how observability helps with reliability and incident response.

OBSERVABILITY ARCHITECTURE



OBSERVE

- ✓ CPU
- ✓ Memory
- ✓ Pods
- ✓ Services
- ✓ Availability
- ✓ Application Metrics
- ✓ Alerts
- ✓ Dashboards

1. DEPLOY PROMETHEUS

Install Prometheus using Helm.

```

helm repo add prometheus-community \
https://prometheus-community.github.io/
helm-charts
helm repo update
helm install prometheus \
prometheus-community/kube-prometheus-stack \
--n monitoring --create-namespace
  
```

2. DEPLOY GRAFANA

Grafana is included in kube-prometheus-stack.

```

kubectl get pods -n monitoring
kubectl get svc -n monitoring
  
```

✓ Access Grafana

```

kubectl port-forward svc/prometheus-grafana \
--n monitoring 3000:80
  
```

Open: <http://localhost:3000>

Default login:
admin / prom-operator

3. VIEW METRICS

Explore Kubernetes and application metrics.

In Grafana, explore:

- Pod CPU and memory usage
- Container restarts
- Node metrics
- Application /metrics endpoint

Example PromQL queries:

```

# Pod CPU usage
rate(container_cpu_usage_seconds_total[5m])

# Pod memory usage
container_memory_usage_bytes

# HTTP request rate (if available)
rate(http_requests_total[5m])
  
```

4. CREATE A DASHBOARD

Use prebuilt dashboards or create your own.

- ✓ Import a Kubernetes dashboard (e.g. ID 315).
- ✓ Create an application dashboard.
- ✓ Visualize CPU, memory, requests, errors and latency.
- ✓ Add useful panels and labels.

5. SET UP ALERTS

Use Alertmanager to send notifications.

```

# Example alert rule
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: example-alerts
  namespace: monitoring
spec:
  groups:
  - name: example
    rules:
    - alert: HighCPUUsage
      expr: rate(container_cpu_usage_seconds_total[5m]) > 0.8
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "High CPU usage on {{ $labels.pod }}"
  
```

6. INVESTIGATE A PROBLEM

Use metrics to find and diagnose issues.

- ✓ Simulate high CPU load and observe the metrics.
- ✓ Find the Pod using the most memory.
- ✓ Identify Pods that are restarting.
- ✓ Check application error rates.
- ✓ Use Grafana and Prometheus to investigate the root cause.
- ✓ Take action to fix the issue.



REAL-WORLD SKILL

Observability helps you detect problems earlier, reduce downtime and keep applications reliable.

KEY CONCEPTS

- Prometheus** - Collects and stores metrics.
- Grafana** - Visualizes metrics with dashboards.
- Alertmanager** - Sends alerts (email, Slack, etc.).
- Metrics** - Numerical data about the system.
- PromQL** - Query language for Prometheus.
- Exporter** - Exposes metrics from applications.

COMMON BEGINNER MISTAKES

- ✗ Not scraping application metrics.
- ✗ Using the wrong namespace.
- ✗ Not setting up alerts.
- ✗ Ignoring resource usage trends.
- ✗ Not checking dashboard labels.
- ✗ Assuming no errors if the app is running.
- ✗ Not investigating anomalies.

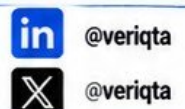
PRACTICE THIS

- ✓ Deploy Prometheus and Grafana.
- ✓ Create a dashboard for your application.
- ✓ Generate load and observe the metrics.
- ✓ Set up an alert for high CPU or memory.
- ✓ Find and fix a problem using metrics.
- ✓ Document what you observed.



NEXT STEP

In the next project, you will centralize application logs and build a complete logging solution for your Kubernetes cluster.



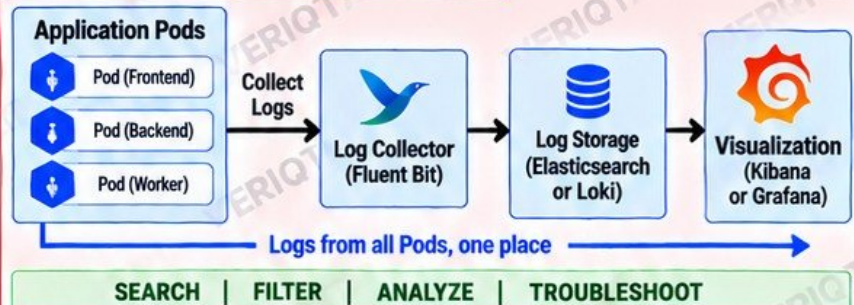
Centralize Application Logs

Collect logs from multiple Pods. Connect kubectl logs with centralized logging concepts and troubleshoot a failing distributed application.

WHAT YOU WILL BUILD

- ✓ Set up a centralized logging stack (e.g. EFK or Loki).
- ✓ Collect logs from multiple Pods and Namespaces.
- ✓ View and search logs in a central dashboard.
- ✓ Correlate logs to troubleshoot a real issue.
- ✓ Understand log structure, labels and metadata.
- ✓ Connect kubectl logs with centralized logging.
- ✓ Learn best practices for logging in Kubernetes.

LOGGING ARCHITECTURE OVERVIEW



1. VIEW POD LOGS (BASICS)

View logs directly from a Pod using kubectl.

```
# View logs from a Pod
kubectl logs <pod-name>

# Follow logs in real time
kubectl logs -f <pod-name>

# View logs from a specific container
kubectl logs <pod-name> -c <container>

# View logs from the last 100 lines
kubectl logs <pod-name> --tail=100
```

2. DEPLOY A LOGGING STACK

Deploy Fluent Bit, Elasticsearch and Kibana (or Loki and Grafana).

Using Helm (EFK example)

```
helm repo add elastic https://helm.elastic.co
helm install elasticsearch elastic/elasticsearch
helm install kibana elastic/kibana
helm install fluent-bit fluent/fluent-bit
```

ALTERNATIVE: LOKI + GRAFANA

```
helm repo add grafana https://grafana.github.io/
helm install loki grafana/loki
helm install promtail grafana/promtail
helm install grafana grafana/grafana
```

3. EXPLORE LOGS IN DASHBOARD

Search and analyze logs in Kibana or Grafana.

KIBANA (ELK)

- Open: `http://<kibana-service>:5601`
- Search logs by Pod name, namespace, labels or keywords.
- Use filters and time range.
- Visualize logs and create dashboards.

GRAFANA (LOKI)

- Open: `http://<grafana-service>:3000`
- Explore logs using LogQL.
- Filter by namespace, pod or container.
- Create useful dashboards and alerts.

4. TROUBLESHOOT A FAILING APP

Use centralized logs to find and fix issues.

```
# Example: search for error logs
level:error

# Filter logs for a specific Pod
kubernetes.pod.name="backend-7d9f"

# Filter by namespace
kubernetes.namespace="production"

# Look for exceptions
"Exception" OR "Traceback" OR "Error"
```

5. USEFUL LOG QUERIES (LOKI)

Common LogQL queries in Grafana.

```
# All logs from a namespace
{namespace="production"}

# Logs from a specific pod
{pod="backend-7d9f"}

# Error logs only
{level="error"}

# Search for a keyword
{namespace="production"} |= "database"

# Count error logs over time
count_over_time({level="error"})[5m]
```

6. ADD CONTEXT AND LABELS

Use labels and structured logging.

- ✓ Include application name, environment and version in log labels.
- ✓ Use structured logs (JSON) where possible.
- ✓ Correlate logs across multiple services using request IDs.
- ✓ Set appropriate log retention.
- ✓ Understand the cost of log storage.



REAL-WORLD SKILL

Centralized logging helps you find the root cause faster, especially in multi-service applications.

KEY CONCEPTS

Fluent Bit - Collects and forwards logs.
Elasticsearch - Stores and indexes logs.
Kibana - Visualizes and searches logs.
Loki - Lightweight log storage.
Grafana - Dashboards and log exploration.
LogQL - Query language for Loki.
Structured Logging - Logs in JSON format.

COMMON BEGINNER MISTAKES

- ✗ Not collecting logs from all namespaces.
- ✗ Missing important labels.
- ✗ Storing logs without retention policy.
- ✗ Using unstructured logs.
- ✗ Not securing the logging stack.
- ✗ Ignoring log volume and cost.
- ✗ Not using logs to troubleshoot real problems.

PRACTICE THIS

- ✓ Deploy a logging stack in your cluster.
- ✓ Generate logs from a sample application.
- ✓ View and search logs in Grafana or Kibana.
- ✓ Find and fix an intentional error.
- ✓ Create a dashboard for your application.
- ✓ Add meaningful labels to your logs.
- ✓ Write a short report on what you discovered.



NEXT STEP

In the next project, you will intentionally break things and practice real troubleshooting scenarios.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

LEARN
BUILD
SHARE
GROW



ENGINEER TODAY
A HIGHER TOMORROW

Kubernetes Failure Lab

Intentionally create CrashLoopBackOff, ImagePullBackOff, Pending Pods, failed probes, bad ConfigMaps, missing Secrets, DNS failures, and Service connectivity problems. Troubleshoot using evidence rather than guessing.

WHAT YOU WILL BUILD

- ✓ Create common Kubernetes failures intentionally.
- ✓ Understand the root cause of each failure.
- ✓ Use kubectl, logs, events and describe to troubleshoot.
- ✓ Learn to read real evidence instead of guessing.
- ✓ Fix the issue and verify the application works.
- ✓ Build troubleshooting skills for production.

COMMON KUBERNETES FAILURES


CrashLoopBackOff
 Container keeps restarting


ImagePullBackOff
 Cannot pull container image


Pending Pods
 Pod cannot be scheduled


Failed Probes
 Liveness or readiness probe failure


Bad ConfigMaps
 Missing or incorrect configuration


Missing Secrets
 Secret not found or incorrect


DNS Failures
 Cannot resolve service name


Service Connectivity
 Cannot reach service or application

1. CRASHLOOPBACKOFF

Create a failing container that exits.

```
apiVersion: v1
kind: Pod
metadata:
  name: crashpod
spec:
  containers:
  - name: app
    image: busybox
    command: ["sh", "-c", "exit 1"]
```

Check and troubleshoot:

```
kubectl get pods
kubectl describe pod crashpod
kubectl logs crashpod --previous
```

2. IMAGEPULLBACKOFF

Use a non-existent image.

```
apiVersion: v1
kind: Pod
metadata:
  name: badimage
spec:
  containers:
  - name: app
    image: myrepo/does-not-exist:latest
```

Check and troubleshoot:

```
kubectl get pods
kubectl describe pod badimage
```

3. PENDING PODS

Request more resources than available.

```
apiVersion: v1
kind: Pod
metadata:
  name: pending-pod
spec:
  containers:
  - name: app
    image: nginx
    resources:
      requests:
        memory: "10Gi"
        cpu: "4"
```

Check and troubleshoot:

```
kubectl get pods
kubectl describe pod pending-pod
kubectl get nodes
```

4. FAILED PROBES

Configure an incorrect probe.

```
apiVersion: v1
kind: Pod
metadata:
  name: probe-fail
spec:
  containers:
  - name: app
    image: nginx
    livenessProbe:
      httpGet:
        path: /wrong
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 5
```

Check and troubleshoot:

```
kubectl describe pod probe-fail
kubectl logs probe-fail
```

5. BAD CONFIGMAPS

Reference a missing key.

```
env:
- name: APP_MODE
  valueFrom:
    configMapKeyRef:
      name: app-config
      key: missing-key
```

Check and troubleshoot:

```
kubectl describe pod <pod-name>
kubectl get configmap
kubectl logs <pod-name>
```

6. MISSING SECRETS

Reference a non-existent secret.

```
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: app-secret
      key: password
```

Check and troubleshoot:

```
kubectl describe pod <pod-name>
kubectl get secret
kubectl logs <pod-name>
```

7. DNS FAILURES

Use a wrong service name.

```
apiVersion: v1
kind: Pod
metadata:
  name: dns-test
spec:
  containers:
  - name: app
    image: busybox
    command: ["sh", "-c", "nslookup wrong-service || true"]
```

Check and troubleshoot:

```
kubectl exec -it dns-test -- nslookup
kubectl get svc
kubectl get endpoints
```

8. SERVICE CONNECTIVITY

Test service-to-service access.

```
# From one pod, test another service
kubectl exec -it <pod> -- sh
```

```
# Test using curl or wget
curl http://my-service:80
```

```
# Check service and endpoints
kubectl get svc
kubectl get endpoints
kubectl get networkpolicies
```

TROUBLESHOOTING WORKFLOW

- 1 Check Pod status (kubectl get pods)
- 2 Describe the resource (kubectl describe)
- 3 Check logs and events
- 4 Identify the root cause
- 5 Make the fix
- 6 Verify the application is working

COMMON MISTAKES

- ✗ Not checking events and logs.
- ✗ Assuming it is a Kubernetes bug.
- ✗ Using the wrong namespace.
- ✗ Forgetting image pull credentials.
- ✗ Misconfigured probes.
- ✗ Ignoring DNS and network policies.
- ✗ Not verifying the fix.

PRACTICE THIS

- ✓ Recreate each failure from scratch.
- ✓ Document the exact error message.
- ✓ Find the root cause using evidence.
- ✓ Apply the fix and verify.
- ✓ Try combining multiple failures.
- ✓ Use your own application (not just sample pods)

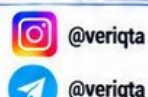
JUNIOR ENGINEER TIP

Real engineers don't guess. They use evidence: status, events, logs and configuration.



NEXT STEP

In the next project, you will build a complete disaster recovery and backup strategy for Kubernetes.



LEARN
BUILD
SHARE
GROW

ENGINEER TODAY
A HIGHER TOMORROW

Capstone: Production-Style Kubernetes Platform

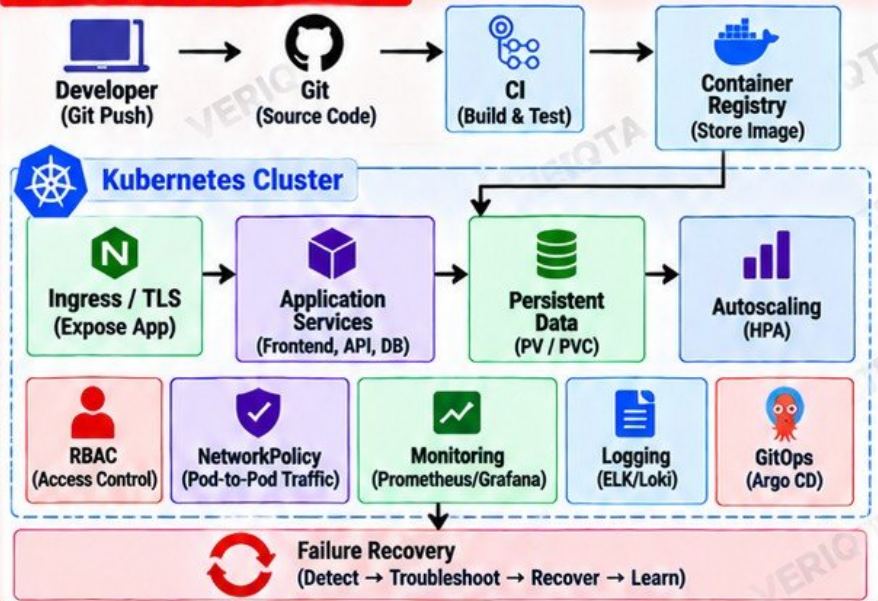
Real Skills
Real Projects
Real Engineers

Combine the notebook into one complete project: Git → CI → registry → Kubernetes → Ingress/TLS → application services → persistent data → autoscaling → RBAC → NetworkPolicy → monitoring/logging → GitOps → failure recovery.

WHAT YOU WILL BUILD

- ✓ A complete, production-style Kubernetes platform.
- ✓ Deploy a real application end-to-end.
- ✓ Implement security, networking, observability and GitOps.
- ✓ Configure autoscaling and persistent data.
- ✓ Practice failure recovery and disaster scenarios.
- ✓ Apply everything you learned in this notebook.

HIGH-LEVEL ARCHITECTURE



1. SOURCE CODE (GIT)

Store the application code in a Git repository.

```
git clone https://github.com/your-org/app.git
cd app
git push origin main
```

4. KUBERNETES DEPLOYMENT

Deploy the application to Kubernetes.

```
kubectl apply -f k8s/
kubectl get pods -n production
```

7. PERSISTENT DATA

Use Persistent Volumes and Claims.

```
kubectl apply -f pvc.yaml
kubectl get pv, pvc
```

10. NETWORK POLICY

Control pod-to-pod traffic.

```
kubectl apply -f networkpolicy.yaml
kubectl describe networkpolicy
```

13. FAILURE RECOVERY

Simulate failures and recover.

```
kubectl describe pod <pod-name>
kubectl logs <pod-name>
# Fix the issue and verify
```

2. CI PIPELINE

Build, test and create a container image.

```
# Example: GitHub Actions
- build application
- run tests
- build Docker image
- push to registry
```

5. INGRESS AND TLS

Expose the application securely.

```
kubectl apply -f ingress.yaml
# Configure TLS with cert-manager
```

8. AUTOSCALING

Configure Horizontal Pod Autoscaler.

```
kubectl apply -f hpa.yaml
kubectl get hpa
kubectl top pods
```

11. MONITORING AND LOGGING

Deploy Prometheus, Grafana and a log stack.

```
kubectl apply -f monitoring/
kubectl port-forward svc/grafana 3000:3000
```

3. CONTAINER REGISTRY

Store the container image.

```
docker build -t myapp:1.0.0 .
docker tag myapp:1.0.0 <registry>/myapp:1.0.0
docker push <registry>/myapp:1.0.0
```

6. APPLICATION SERVICES

Run frontend, backend and database.

```
kubectl get svc
kubectl get endpoints
kubectl get pods
```

9. SECURITY (RBAC)

Create ServiceAccounts, Roles and Bindings.

```
kubectl apply -f rbac.yaml
kubectl auth can-i list pods
--as=system:serviceaccount:app:default
```

12. GITOPS (ARGO CD)

Keep the cluster in sync with Git.

```
kubectl apply -f argocd/
# Change Git and sync in Argo CD
```

KEY OUTCOMES

- ✓ A complete, production-ready project.
- ✓ End-to-end Kubernetes skills.
- ✓ Real-world troubleshooting experience.
- ✓ The confidence to work on real systems.



JUNIOR ENGINEER TIP

This project is not just about running an application. It's about thinking like a production engineer — secure, observable, scalable and recoverable.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



ENGINEER TODAY
A HIGHER TOMORROW

LEARN
BUILD
SHARE
GROW